

```

1 class Solution {
2 public:
3     vector<int> twoSum(vector<int>& nums, int target) {
4         map<int,int> st;
5         int len=nums.size();
6         for (int i=0;i<len;i++){
7             if (st.find(target-nums[i])==st.end())
8                 st[nums[i]]=i;
9             else
10                 return {st[target-nums[i]],i};
11         }
12         return {-1,-1};
13     }
14 };

```

AI写代码cpp运行

2. 两数相加

给你两个非空的链表,表示两个非负的整数。它们每位数字都是按照 逆序 的方式存储的,并且每个节点只能存储 一位 数字。
 请你将两个数相加,并以相同形式返回一个表示和的链表。
 你可以假设除了数字 0 之外,这两个数都不会以 0 开头。

```

1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * };
10 */
11 class Solution {
12 public:
13     ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
14         ListNode *head=NULL;
15         ListNode *tail=NULL;
16         int c=0;
17         int sum=0;
18         while(l1!=NULL||l2!=NULL){
19             int num1=l1?l1->val:0;
20             int num2=l2?l2->val:0;
21             sum=num1+num2+c;
22             if(!head)
23                 head=tail=new ListNode(sum%10);
24             else{
25                 tail->next=new ListNode(sum%10);
26                 tail=tail->next;
27             }
28             c=sum/10;
29             if(l1)
30                 l1=l1->next;
31             if(l2)
32                 l2=l2->next;
33         }
34         if(c>0)
35             tail->next=new ListNode(c);
36         return head;
37     }
38 };

```

AI写代码cpp运行

3. 无重复字符的最长子串

```

1 class Solution {
2 public:
3     int lengthOfLongestSubstring(string s) {
4         set<char> st;
5         int len=s.size();

```

```

6         int ans=0;
7         int r=-1;
8         for(int i=0;i<len;i++){
9             if (i!=0)
10                st.erase(s[i-1]);
11                while(r+1<len&&st.find(s[r+1])==st.end())
12                    st.insert(s[++r]);
13                ans=max(ans,r-i+1);
14            }
15        return ans;
16    }
17 };

```

AI写代码cpp运行

4. 寻找两个正序数组的 中位数

给定两个大小分别为 m 和 n 的正序（从小到大）数组 $nums1$ 和 $nums2$ 。请你找出并返回这两个正序数组的中位数。算法的时间复杂度应该为 $O(\log(m+n))$ 。

```

1 class Solution {
2 public:
3     double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
4         int tot=nums1.size()+nums2.size();
5         if (tot%2==0){
6             int left=find(nums1,0,nums2,0,tot/2);
7             int right=find(nums1,0,nums2,0,tot/2+1);
8             return (left+right)/2.0;
9         }
10        else return (double)find(nums1,0,nums2,0,tot/2+1);
11    }
12    int find(vector<int> &nums1,int i,vector<int> &nums2,int j,int k){
13        if (nums1.size()-i>nums2.size()-j)
14            return find(nums2,j,nums1,i,k);
15        if (k==1){
16            if (nums1.size()==i)
17                return nums2[j];
18            else return min(nums1[i],nums2[j]);
19        }
20        if (nums1.size()==i)
21            return nums2[j+k-1];
22        int si=min((int)nums1.size(),i+k/2);
23        int sj=j+k/2;
24        if (nums1[si-1]>nums2[sj-1])
25            return find(nums1,i,nums2,sj,k-(sj-j));
26        else return find(nums1,si,nums2,j,k-(si-i));
27    }
28 };

```

AI写代码cpp运行

5.最长回文子串

```

1 class Solution {
2 public:
3     string longestPalindrome(string s) {
4         int n=s.size();
5         string res;
6         for(int i=0;i<n;i++){
7             int l=i-1,r=i+1;
8             while(l>=0&&r<s.size()&&s[l]==s[r])
9                 l--,r++;
10            if (r-l>res.size())
11                res=s.substr(l+1,r-l-l);
12            l=i,r=i+1;
13            while(l>=0&&r<s.size()&&s[l]==s[r])
14                l--,r++;
15            if (r-l>res.size())
16                res=s.substr(l+1,r-l-l);
17        }
18        return res;
19    }
20 };

```

10. 正则表达式匹配

给你一个字符串 s 和一个字符规律 p ，请你来实现一个支持 '.' 和 '*' 的正则表达式匹配。

'.' 匹配任意单个字符

'*' 匹配零个或多个前面的那一个元素

所谓匹配，是要涵盖 整个 字符串 s 的，而不是部分字符串。

```

1  class Solution {
2  public:
3      bool f[35][35];
4      bool isMatch(string s, string p) {
5          int n=s.size();
6          int m=p.size();
7          s=' '+s;
8          p=' '+p;
9          f[0][0]=1;
10         for(int i=0;i<=n;i++){
11             for(int j=1;j<=m;j++){
12                 if (j+1<=m&&p[j+1]=='*'){
13                     continue;
14                 }
15                 if (i&&p[j]!='*')
16                     f[i][j]=f[i-1][j-1]&&(s[i]==p[j]||p[j]=='.' );
17                 else if (p[j]=='*')
18                     f[i][j]=f[i][j-2]|| (i&&f[i-1][j])&&(s[i]==p[j-1]||p[j-1]=='.' );
19             }
20         }
21         return f[n][m];
22     };

```

AI写代码cpp运行

11. 盛最多水的容器

给定一个长度为 n 的整数数组 $height$ 。有 n 条垂线，第 i 条线的两个端点是 $(i, 0)$ 和 $(i, height[i])$ 。

找出其中的两条线，使得它们与 x 轴共同构成的容器可以容纳最多的水。

返回容器可以储存的最大水量。

说明： 你不能倾斜容器。

```

1  class Solution {
2  public:
3      int maxArea(vector<int>& height) {
4          int i=0;
5          int j=height.size()-1;
6          int maxa=min(height[i],height[j])*(j-i);
7          while(i<j){
8              int a=min(height[i],height[j])*(j-i);
9              maxa=max(maxa,a);
10             if(height[i]<height[j])
11                 i++;
12             else
13                 j--;
14         }
15         return maxa;
16     }
17 };

```

AI写代码cpp运行

15. 三数之和

给你一个包含 n 个整数的数组 $nums$ ，判断 $nums$ 中是否存在三个元素 a ， b ， c ，使得 $a + b + c = 0$ ？请你找出所有和为 0 且不重复的三元组。

注意：答案中不可以包含重复的三元组。

```

1  class Solution {
2  public:
3      vector<vector<int>> threeSum(vector<int>& nums) {
4          set<vector<int>> resbuf;
5          int n=nums.size();

```

```

6         sort(nums.begin(), nums.end());
7         for(int i=0; i<n-1; i++){
8             if (i>0&&nums[i]==nums[i-1])
9                 continue;
10            int l=i+1, r=n-1;
11            while(l<r){
12                if (nums[l]+nums[i]+nums[r]>0)
13                    r--;
14                else if (nums[l]+nums[i]+nums[r]<0)
15                    l++;
16                else if (nums[l]+nums[i]+nums[r]==0){
17                    resbuf.insert({nums[i], nums[l], nums[r]});
18                    l++;
19                }
20            }
21        }
22        vector<vector<int>> > res;
23        for(auto node: resbuf)
24            res.push_back(node);
25        return res;
26    }
27 };
    AI写代码cpp运行

```

17. 电话号码的字母组合

给定一个仅包含数字 2 – 9 的字符串，返回所有它能表示的字母组合。答案可以按 任意顺序 返回。
给出数字到字母的映射如下（与电话按键相同）。注意 1 不对应任何字母。

```

1  class Solution {
2  public:
3      set<char> sc[11];
4      int n;
5      vector<string> ans;
6      string digits;
7      void dfs(int i, string res){
8          if (i==n){
9              ans.push_back(res);
10             return;
11         }
12         for (char ch:sc[digits[i]-'0'])
13             dfs(i+1, res+ch);
14         return;
15     }
16     vector<string> letterCombinations(string digits) {
17         if (digits=="")
18             return ans;
19         this->digits=digits;
20         sc[2]={ 'a', 'b', 'c' };
21         sc[3]={ 'd', 'e', 'f' };
22         sc[4]={ 'g', 'h', 'i' };
23         sc[5]={ 'j', 'k', 'l' };
24         sc[6]={ 'm', 'n', 'o' };
25         sc[7]={ 'p', 'q', 'r', 's' };
26         sc[8]={ 't', 'u', 'v' };
27         sc[9]={ 'w', 'x', 'y', 'z' };
28         n=digits.size();
29
30         string res="";
31         dfs(0, res);
32         return ans;
33     }
34 };
    AI写代码cpp运行

```

19. 删除链表的倒数第 N 个结点

```

1  class Solution {
2  public:
3      ListNode* removeNthFromEnd(ListNode* head, int n) {
4

```

```

18         ListNode* dummy=new ListNode(0,head);
19         ListNode *first=head;
20         for(int i=0;i<n;i++)
21             first=first->next;
22         ListNode *second=dummy;
23         while(first!=NULL){
24             first=first->next;
25             second=second->next;
26         }
27         ListNode *p=second->next;
28         second->next=p->next;
29         delete p;
30         head=dummy->next;
31         return head;
32     }
33 };
34 AI写代码cpp运行

```

20. 有效的括号

```

1  class Solution {
2  public:
3      bool check(char c1,char c2){
4          if (c1=='(' && c2==')' || c1=='{' && c2=='}' || c1=='[' && c2==']')
5              return true;
6          else return false;
7      }
8      bool isValid(string s) {
9          int i=0;
10         stack<char> stk;
11         for (int i=0;i<s.size();i++){
12             if (!stk.empty() && check(stk.top(),s[i]))
13                 stk.pop();
14             else
15                 stk.push(s[i]);
16         }
17         if (stk.empty())
18             return true;
19         else
20             return false;
21     }
22 };
23 AI写代码cpp运行

```

21. 合并两个有序链表

```

1  class Solution {
2  public:
3      ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
4          ListNode *dummy=new ListNode(-1);
5          ListNode *tail=dummy;
6          while(list1 && list2){
7              if (list1->val<list2->val){
8                  tail->next=list1;
9                  tail=tail->next;
10                 list1=list1->next;
11             }
12             else {
13                 tail->next=list2;
14                 tail=tail->next;
15                 list2=list2->next;
16             }
17         }
18         if (list1!=NULL)
19             tail->next=list1;
20         if (list2!=NULL)
21             tail->next=list2;
22         return dummy->next;
23     }
24 };

```

22. 括号生成

```

1  class Solution {
2  public:
3      string str;
4      vector<string> res;
5      int len;
6      void dfs(int left,int right){
7          if (len*2==str.size()){
8              res.push_back(str);
9              return ;
10         }
11         if (left<len){
12             str.push_back('(');
13             dfs(left+1,right);
14             str.pop_back();
15         }
16         if (right<left){
17             str.push_back(')');
18             dfs(left,right+1);
19             str.pop_back();
20         }
21     }
22     vector<string> generateParenthesis(int n) {
23         len=n;
24         dfs(0,0);
25         return res;
26     }
27 };

```

AI写代码cpp运行

23. 合并K个升序链表

```

1  class Solution {
2  public:
3      struct cmp{
4          bool operator()(ListNode *a,ListNode*b){
5              return a->val>b->val;
6          }
7      };
8      ListNode* mergeKLists(vector<ListNode*>& lists) {
9          priority_queue<ListNode *,vector<ListNode *>,cmp> pq;
10         ListNode *dummy=new ListNode(-1,NULL);
11         ListNode *tail=dummy;
12         for(auto list:lists)
13             if (list)
14                 pq.push(list);
15         while(!pq.empty()){
16             auto p=pq.top();
17             pq.pop();
18             tail->next=p;
19             tail=tail->next;
20             if (p->next!=NULL){
21                 pq.push(p->next);
22                 p=p->next;
23             }
24         }
25         return dummy->next;
26     }
27 };

```

AI写代码cpp运行

31. 下一个排列

```

1  class Solution {
2  public:

```

```

3 void nextPermutation(vector<int>& nums) {
4     int n=nums.size();
5     int k=n-1;
6     while(k>0&&nums[k-1]>=nums[k])
7         k--;
8     if (k<=0){
9         reverse(nums.begin(),nums.end());
10        return;
11    }
12    int t=k;
13    while(t<n&&nums[t]>nums[k-1])
14        t++;
15    swap(nums[t-1],nums[k-1]);
16    sort(nums.begin()+k,nums.end());
17    return;
18 }
19 };

```

AI写代码cpp运行

32. 最长有效括号

```

1 class Solution {
2 public:
3     int longestValidParentheses(string s) {
4         stack<int> stk;
5         int ans=0;
6         stk.push(-1);
7         int maxlen=s.size();
8         for (int i=0;i<maxlen;i++){
9             if (s[i]=='(')
10                stk.push(i);
11            else{
12                stk.pop();
13                if (stk.empty())
14                    stk.push(i);
15                else
16                    ans=max(ans,i-stk.top());
17            }
18        }
19        return ans;
20    }
21 };

```

AI写代码cpp运行

33. 搜索旋转排序数组

整数数组 *nums* 按升序排列，数组中的值 互不相同。

在传递给函数之前，*nums* 在预先未知的某个下标 *k* ($0 \leq k < \text{nums.length}$) 上进行了 旋转，使数组变为 $[\text{nums}[k], \text{nums}[k+1], \dots, \text{nums}[n-1], \text{nums}[0], \dots, \text{nums}[k-1]]$ （下标从 0 开始 计数）。例如， $[0, 1, 2, 4, 5, 6, 7]$ 在下标 3 处经旋转后可能变为 $[4, 5, 6, 7, 0, 1, 2]$ 。

给你 旋转后 的数组 *nums* 和一个整数 *target*，如果 *nums* 中存在这个目标值 *target*，则返回它的下标，否则返回 -1 。

你必须设计一个时间复杂度为 $O(\log n)$ 的算法解决此问题。

```

1 class Solution {
2 public:
3     int search(vector<int>& nums, int target) {
4         if (nums.empty()) return -1;
5         int l = 0, r = nums.size() - 1;
6         while (l < r) {
7             int mid = l + r + 1 >> 1;
8             if (nums[mid] >= nums[0]) l = mid;
9             else r = mid - 1;
10        }
11
12        if (target >= nums[0]) l = 0;
13        else l = r + 1, r = nums.size() - 1;
14
15        while (l < r) {
16            int mid = l + r >> 1;
17            if (nums[mid] >= target) r = mid;
18            else l = mid + 1;
19        }
20        return l;
21    }
22 };

```

```

19     }
20
21     if (nums[r] == target) return r;
22     return -1;
23 }
24 };

```

AI写代码cpp运行

34. 在排序数组中查找元素的第一个和最后一个位置

```

1  class Solution {
2  public:
3      vector<int> searchRange(vector<int>& nums, int target) {
4          if(nums.size()==0)
5              return {-1,-1};
6          int l=0;
7          int r=nums.size()-1;
8          while(l<r){
9              int m=(l+r)/2;
10             if (nums[m]>=target)
11                 r=m;
12             else l=m+1;
13         }
14         int L=l;
15         if(nums[L]!=target)
16             return {-1,-1};
17         l=0;
18         r=nums.size()-1;
19         while(l<r){
20             int m=(l+r+1)/2;
21             if(nums[m]<=target) l=m;
22             else r=m-1;
23         }
24
25         int R=r;
26         return {L,R};
27     }
28 };

```

AI写代码cpp运行

39. 组合总和

给你一个无重复元素的整数数组 *candidates* 和一个目标整数 *target*，找出 *candidates* 中可以使数字和为目标数 *target* 的所有不同组合，并以列表形式按任意顺序返回这些组合。

candidates 中的同一个数字可以无限制重复被选取。如果至少一个数字的被选数量不同，则两种组合是不同的。

对于给定的输入，保证和为 *target* 的不同组合数少于 150 个。

```

1  class Solution {
2  public:
3      vector<int> candidates;
4      vector<vector<int>> res;
5      vector<int> tempres;
6      int n;
7      void dfs(int sum,int i){
8          if (i>=n)
9              return;
10
11          if (sum==0){
12              res.push_back(tempres);
13              return;
14          }
15
16          if (sum>=candidates[i]){
17              tempres.push_back(candidates[i]);
18              dfs(sum-candidates[i],i);
19              tempres.pop_back();
20              dfs(sum,i+1);
21          }
22          else dfs(sum,i+1);
23      }
24  };

```



```

24     vector<vector<int>> combinationSum(vector<int>& candidates, int target) {
25         sort(candidates.begin(), candidates.end(), greater<int>());
26         n=candidates.size();
27         this->candidates=candidates;
28         dfs(target, 0);
29         return res;
30     }
31 };

```

AI写代码cpp运行

42. 接雨水

给定 n 个非负整数表示每个宽度为 1 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

```

1  class Solution {
2  public:
3      int trap(vector<int>& height) {
4          int ans=0;
5          stack<int> stk;
6          int maxlen=height.size();
7          for (int i=0;i<maxlen;i++){
8              while(!stk.empty()&&height[i]>height[stk.top()]){
9                  int h=stk.top();
10                 stk.pop();
11                 if (stk.empty())
12                     break;
13                 int left=stk.top();
14                 int w=i-left-1;
15                 int ht=min(height[left],height[i])-height[h];
16                 ans+=w*ht;
17             }
18             stk.push(i);
19         }
20         return ans;
21     }
22 };

```

AI写代码cpp运行

46. 全排列

```

1  class Solution {
2  vector<vector<int>> >res;
3  bool st[10];
4  int len;
5  vector<int> vct;
6  vector<int> nm;
7  public:
8      void dfs(int n){
9          if (n==len)
10             res.push_back(vct);
11         for (int i=0;i<len;i++){
12             if(!st[i]){
13                 st[i]=true;
14                 vct.push_back(nm[i]);
15                 dfs(n+1);
16                 st[i]=false;
17                 vct.pop_back();
18             }
19         }
20     }
21     vector<vector<int>> permute(vector<int>& nums) {
22         len=nums.size();
23         nm=nums;
24         dfs(0);
25         return res;
26     }
27 };

```

AI写代码cpp运行

48. 旋转图像

给定一个 $n \times n$ 的二维矩阵 *matrix* 表示一个图像。请你将图像顺时针旋转 90 度。
你必须在 原地 旋转图像，这意味着你需要直接修改输入的二维矩阵。请不要 使用另一个矩阵来旋转图像。

```
1 class Solution {
2 public:
3     void rotate(vector<vector<int>>& matrix) {
4         int n=matrix.size();
5         for(int i=0;i<n/2;i++){
6             for(int j=0;j<n;j++){
7                 swap(matrix[i][j],matrix[n-i-1][j]);
8             }
9             for(int i=0;i<n;i++){
10                for(int j=0;j<i;j++){
11                    swap(matrix[i][j],matrix[j][i]);
12                }
13            }
14        }
15    };
16 }
```

AI写代码cpp运行

49. 字母异位词分组

给你一个字符串数组，请你将 字母异位词 组合在一起。可以按任意顺序返回结果列表。
字母异位词 是由重新排列源单词的字母得到的一个新单词，所有源单词中的字母通常恰好只用一次。

```
1 class Solution {
2 public:
3     map<map<char,int>,vector<string>> resbuf;
4     vector<vector<string>> groupAnagrams(vector<string>& strs) {
5         for(auto str:str){
6             map<char,int> tempset;
7             for(auto ch:str)
8                 tempset[ch]++;
9             resbuf[tempset].push_back(str);
10        }
11        vector<vector<string> > res;
12        for(auto node:resbuf){
13            vector<string> resbuf;
14            for(auto str:node.second)
15                resbuf.push_back(str);
16            res.push_back(resbuf);
17        }
18        return res;
19    }
20 }
```

AI写代码cpp运行

53. 最大子数组和

```
1 class Solution {
2 public:
3     int maxSubArray(vector<int>& nums) {
4         int len=nums.size();
5         vector<int> dp(len);
6         dp[0]=nums[0];
7         for (int i=1;i<len;i++){
8             if (dp[i-1]>=0)
9                 dp[i]=dp[i-1]+nums[i];
10            else
11                dp[i]=nums[i];
12        }
13        int res=-0x3f3f3f3f;
14        for (int i=0;i<len;i++)
15            res=max(res,dp[i]);
16        return res;
17    }
18 }
```

AI写代码cpp运行

55. 跳跃游戏

给定一个非负整数数组 $nums$ ，你最初位于数组的 第一个下标。

数组中的每个元素代表你在该位置可以跳跃的最大长度。

判断你是否能够到达最后一个下标。

```
1  class Solution {
2  public:
3      bool canJump(vector<int>& nums) {
4          int n=nums.size();
5          int rightmost=0;
6          for(int i=0;i<=rightmost;i++){
7              rightmost=max(rightmost,i+nums[i]);
8              if (rightmost>=n-1)
9                  return true;
10         }
11         return false;
12     }
13 };
AI写代码cpp运行
```

56. 合并区间

以数组 $intervals$ 表示若干个区间的集合，其中单个区间为 $intervals[i] = [start_i, end_i]$ 。请你合并所有重叠的区间，并返回 一个不重叠的区间数组覆盖输入中的所有区间。

```
1  class Solution {
2  public:
3      vector<vector<int>> merge(vector<vector<int>>& intervals) {
4          sort(intervals.begin(),intervals.end());
5          int st=intervals[0][0],ed=intervals[0][1];
6          vector<vector<int>> res;
7          int n=intervals.size();
8          for(int i=0;i<n;i++){
9              if (i&&intervals[i][0]>ed){
10                 res.push_back({st,ed});
11                 st=intervals[i][0];
12                 ed=intervals[i][1];
13             }
14             if (intervals[i][0]<=ed&&intervals[i][1]>=ed)
15                 ed=intervals[i][1];
16         }
17         res.push_back({st,ed});
18         return res;
19     }
20 };
AI写代码cpp运行
```

72. 编辑距离

给你两个单词 $word_1$ 和 $word_2$ ，请返回将 $word_1$ 转换成 $word_2$ 所使用的最少操作数。

可以对一个单词进行如下三种操作：

插入一个字符

删除一个字符

替换一个字符

```
1  class Solution {
2  public:
3      int dp[501][501];
4      int minDistance(string word1, string word2) {
5          int len1=word1.size();
6          int len2=word2.size();
7          if(len1*len2==0)
8              return len1+len2;
9          for(int i=0;i<=len1;i++)
10             dp[i][0]=i;
11          for(int j=0;j<=len2;j++)
12             dp[0][j]=j;
13          for(int i=1;i<=len1;i++)
14             for(int j=1;j<=len2;j++)
15                 dp[i][j]=min(dp[i-1][j],dp[i][j-1],dp[i-1][j-1]+(word1[i-1]!=word2[j-1]));
16         return dp[len1][len2];
17     }
18 };
AI写代码cpp运行
```

```

15         for(int j=1;j<=len2;j++){
16             if (word1[i-1]==word2[j-1])
17                 dp[i][j]=dp[i-1][j-1];
18             else
19                 dp[i][j]=min(dp[i-1][j],min(dp[i][j-1],dp[i-1][j-1]))+1;
20         }
21         return dp[len1][len2];
22     }
};
AI写代码cpp运行

```

75. 荷兰国旗

```

1  class Solution {
2  public:
3      void sortColors(vector<int>& nums) {
4          for(int i=0,j=0,k=nums.size()-1;i<=k;){
5              if (nums[i]==0)
6                  swap(nums[i++],nums[j++]);
7              else if (nums[i]==2)
8                  swap(nums[i],nums[k--]);
9              else i++;
10         }
11     }
12 };
AI写代码cpp运行

```

76. 最小覆盖子串

给你一个字符串 s 、一个字符串 t 。返回 s 中涵盖 t 所有字符的最小子串。如果 s 中不存在涵盖 t 所有字符的子串，则返回空字符串 $""$ 。

```

1  class Solution {
2  public:
3      string minWindow(string s, string t) {
4          map<char,int> hs,ht;
5          for(auto c:t)
6              ht[c]++;
7          string res;
8          int n=s.size();
9          int cnt=0;
10         for(int i=0,j=0;i<n;i++){
11             hs[s[i]]++;
12             if (hs[s[i]]<=ht[s[i]])
13                 cnt++;
14             while(hs[s[j]]>ht[s[j]])
15                 hs[s[j++]]--;
16             if (cnt==t.size())
17                 if (res.empty()||i-j+1<res.size())
18                     res=s.substr(j,i-j+1);
19         }
20         return res;
21     }
22 };
AI写代码cpp运行

```

78. 子集

给你一个整数数组 $nums$ ，数组中的元素互不相同。返回该数组所有可能的子集（幂集）。
解集不能包含重复的子集。你可以按任意顺序返回解集。

```

1  class Solution {
2  public:
3      vector<vector<int>> subsets(vector<int>& nums) {
4          vector<vector<int>> res;
5          int n=nums.size();
6          for(int mask=0;mask<(1<<n);mask++){
7              vector<int> tempres;
8              for(int i=0;i<n;i++)
9                  if (mask>>i&1)
10                     tempres.push_back(nums[i]);

```

```

11         res.push_back(tempres);
12     }
13     return res;
14 }
15 };

```

AI写代码cpp运行

79. 单词搜索

给定一个 $m \times n$ 二维字符网格 *board* 和一个字符串单词 *word*。如果 *word* 存在于网格中，返回 true；否则，返回 false。

单词必须按照字母顺序，通过相邻的单元格内的字母构成，其中“相邻”单元格是那些水平相邻或垂直相邻的单元格。同一个单元格内的字母不允许被重复使

```

1  class Solution {
2  public:
3      bool st[10][10];
4      string word;
5      vector<vector<char>> board;
6      int dx[4]={-1,0,1,0};
7      int dy[4]={0,1,0,-1};
8      int m,n;
9      bool dfs(int i,int j,int pos){
10         if (pos==word.size())
11             return true;
12         for(int k=0;k<4;k++){
13             int px=i+dx[k];
14             int py=j+dy[k];
15             if (px<0||px>=m||py<0||py>=n)
16                 continue;
17             if (st[px][py])
18                 continue;
19             if (word[pos]==board[px][py]){
20                 st[px][py]=1;
21                 if (dfs(px,py,pos+1))
22                     return true;
23                 st[px][py]=0;
24             }
25         }
26         return false;
27     }
28     bool exist(vector<vector<char>>& board, string word) {
29         m=board.size();
30         n=board[0].size();
31         this->word=word;
32         this->board=board;
33         for(int i=0;i<m;i++){
34             for(int j=0;j<n;j++){
35                 if (board[i][j]==word[0]){
36                     st[i][j]=1;
37                     if (dfs(i,j,1))
38                         return true;
39                     st[i][j]=0;
40                 }
41             }
42         }
43         return false;
44     }
45 };

```

AI写代码cpp运行

84. 柱状图中最大的矩形

给定 n 个非负整数，用来表示柱状图中各个柱子的高度。每个柱子彼此相邻，且宽度为 1。

求在该柱状图中，能够勾勒出来的矩形的最大面积。

```

1  class Solution {
2  public:
3      int largestRectangleArea(vector<int>& heights) {
4          int n=heights.size();
5          vector<int> left(n),right(n);
6          stack<int> stk;
7

```

```

8   for(int i=0;i<n;i++){
9       while(stk.size() && heights[stk.top()] >= heights[i])
10          stk.pop();
11       if (stk.empty())
12          left[i] = -1;
13       else left[i] = stk.top();
14       stk.push(i);
15   }
16   while(stk.size())
17       stk.pop();
18   for(int i=n-1;i>=0;i--){
19       while(stk.size() && heights[stk.top()] >= heights[i])
20          stk.pop();
21       if (stk.empty())
22          right[i] = n;
23       else right[i] = stk.top();
24       stk.push(i);
25   }
26   int res=0;
27   for(int i=0;i<n;i++){
28       res = max(res, (right[i] - left[i] - 1) * heights[i]);
29   }
30   return res;
};

```

AI写代码cpp运行

85. 最大矩形

给定一个仅包含 0 和 1、大小为 $rows \times cols$ 的二维二进制矩阵，找出只包含 1 的最大矩形，并返回其面积。

```

1  class Solution {
2  public:
3      int largestRectangleArea(vector<int>& heights) {
4          int n = heights.size();
5          vector<int> left(n), right(n);
6          stack<int> stk;
7          for(int i=0;i<n;i++){
8              while(stk.size() && heights[stk.top()] >= heights[i])
9                  stk.pop();
10             if (stk.empty())
11                 left[i] = -1;
12             else left[i] = stk.top();
13             stk.push(i);
14         }
15         while(stk.size())
16             stk.pop();
17         for(int i=n-1;i>=0;i--){
18             while(stk.size() && heights[stk.top()] >= heights[i])
19                 stk.pop();
20             if (stk.empty())
21                 right[i] = n;
22             else right[i] = stk.top();
23             stk.push(i);
24         }
25         int res=0;
26         for(int i=0;i<n;i++){
27             res = max(res, (right[i] - left[i] - 1) * heights[i]);
28         }
29         return res;
30     }
31     int maximalRectangle(vector<vector<char>>& matrix) {
32         if(matrix.empty() || matrix[0].empty())
33             return 0;
34         int n = matrix.size();
35         int m = matrix[0].size();
36         vector<vector<int>> h(n, vector<int>(m));
37         for(int i=0;i<n;i++){
38             for(int j=0;j<m;j++){
39                 if (matrix[i][j] == '1'){
40                     if (i)
41                         h[i][j] = h[i-1][j] + 1;
42                     else h[i][j] = 1;
43                 }
44             }
45         }
46     }
47 };

```

```

42         }
43     }
44 }
45 int res=0;
46 for(int i=0;i<n;i++)
47     res=max(res,largestRectangleArea(h[i]));
48 return res;
49 }
50 };

```

AI写代码cpp运行

94. 二叉树的中序遍历

给定一个二叉树的根节点 *root* , 返回 它的 中序 遍历。

```

1  /**
2   * Definition for a binary tree node.
3   * struct TreeNode {
4   *     int val;
5   *     TreeNode *left;
6   *     TreeNode *right;
7   *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
8   *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
9   *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
10  * };
11  */
12 class Solution {
13 public:
14     void inorder(TreeNode *root,vector<int> &res){
15         if (root==NULL)
16             return;
17         inorder(root->left,res);
18         res.push_back(root->val);
19         inorder(root->right,res);
20     }
21     vector<int> inorderTraversal(TreeNode* root) {
22         vector<int> res;
23         inorder(root,res);
24         return res;
25     }
26 };

```

AI写代码cpp运行

96. 不同的二叉搜索树

给你一个整数 *n* , 求恰由 *n* 个节点组成且节点值从 1 到 *n* 互不相同的二叉搜索树有多少种? 返回满足题意的二叉搜索树的种数。

```

1  class Solution {
2  public:
3      int numTrees(int n) {
4          vector<int> f(25,0);
5          f[0]=0;
6          f[1]=1;
7          f[2]=2;
8          if (n<3)
9              return f[n];
10         f[0]=1;
11         for(int i=3;i<=n;i++){
12             for(int j=0;j<=i-1;j++)
13                 f[i]+=f[j]*f[i-j-1];
14         }
15         return f[n];
16     }
17 };

```

AI写代码cpp运行

98.验证二叉搜索树

```

1  class Solution {
2  public:
3

```

```

3   long long pre=LONG_MIN;
4   bool isValidBST(TreeNode* root) {
5       if(root==NULL)
6           return true;
7       if (!isValidBST(root->left))
8           return false;
9       if(root->val<=pre)
10          return false;
11          pre=root->val;
12          return isValidBST(root->right);
13      }
14  };

```

AI写代码cpp运行

101.对称二叉树

```

1   class Solution {
2   public:
3       bool dfs(TreeNode* p, TreeNode* q) {
4           if (p==NULL&&q==NULL)
5               return true;
6           else if(p==NULL||q==NULL)
7               return false;
8           else if (p->val!=q->val)
9               return false;
10          else return (dfs(p->left,q->right)&&dfs(p->right,q->left));
11      }
12
13      bool isSymmetric(TreeNode* root) {
14          return dfs(root,root);
15      }
16  };

```

AI写代码cpp运行

102. 二叉树的层序遍历

```

1   class Solution {
2   public:
3       vector<vector<int>> levelOrder(TreeNode* root) {
4           vector<vector<int>> res;
5           if (root==NULL)
6               return res;
7           queue<TreeNode *>q;
8           q.push(root);
9           while(!q.empty()){
10              int qsize=q.size();
11              res.push_back(vector<int>());
12              for(int i=0;i<qsize;i++){
13                  auto node=q.front();
14                  q.pop();
15                  res.back().push_back(node->val);
16                  if (node->left!=NULL)
17                      q.push(node->left);
18                  if (node->right!=NULL)
19                      q.push(node->right);
20              }
21          }
22          return res;
23      }
24  };

```

AI写代码cpp运行

104. 二叉树的最大深度

```

1   class Solution {
2   public:
3       int maxDepth(TreeNode* root) {
4           if (root!=NULL)
5               return max(maxDepth(root->left),maxDepth(root->right))+1;
6           else return 0;

```



```

/
8
};
AI写代码cpp运行

```

105. 从前序与中序遍历序列构造二叉树

```

1  class Solution {
2  public:
3      unordered_map<int,int> num2idx;
4      vector<int> preorder;
5      vector<int> inorder;
6      int n;
7      TreeNode* buildTree(int pl,int pr,int il,int ir){
8          if (pl>pr)
9              return NULL;
10         int rval=preorder[pl];
11         TreeNode* root=new TreeNode(rval);
12         int rootpos=num2idx[rval];
13         int len=rootpos-il;
14         root->left=buildTree(pl+1,pl+len,il,rootpos-1);
15         root->right=buildTree(pl+len+1,pr,rootpos+1,ir);
16         return root;
17     }
18     TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
19         this->preorder=preorder;
20         this->inorder=inorder;
21         n=preorder.size();
22         for(int i=0;i<inorder.size();i++)
23             num2idx[inorder[i]]=i;
24         TreeNode* root=buildTree(0,n-1,0,n-1);
25         return root;
26     }
27 };
AI写代码cpp运行

```

114. 二叉树展开为链表

```

1  class Solution {
2  public:
3      void flatten(TreeNode* &root) {
4          while(root){
5              TreeNode* p=root->left;
6              if (p){
7                  while(p->right)
8                      p=p->right;
9                  p->right=root->right;
10                 root->right=root->left;
11                 root->left=NULL;
12             }
13             root=root->right;
14         }
15     }
16 };
AI写代码cpp运行

```

121. 买卖股票的最佳时机

给定一个数组 *prices*，它的第 *i* 个元素 *prices[i]* 表示一支给定股票第 *i* 天的价格。

你只能选择某一天买入这只股票，并选择在未来的某一个不同的日子 卖出该股票。设计一个算法来计算你能获取的最大利润。

返回你可以从这笔交易中获取的最大利润。如果你不能获取任何利润，返回 0。

```

1  class Solution {
2  public:
3      int maxProfit(vector<int>& prices) {
4          stack<int> stk;
5          int n=prices.size();
6          int res=0;
7          for(int i=0;i<n;i++){
8              if (stk.empty()||prices[i]<stk.top())
9                  stk.push(prices[i]);

```

```

10         res=max(res,prices[i]-stk.top());
11     }
12     return res;
13 }
14 };

```

AI写代码cpp运行

124. 二叉树中的最大路径和

路径 被定义为一条从树中任意节点出发，沿父节点-子节点连接，达到任意节点的序列。同一个节点在一条路径序列中 至多出现一次 。该路径 至少包含一个 经过根节点。

路径和 是路径中各节点值的总和。

给你一个二叉树的根节点 *root* ，返回其 最大路径和 。

```

1  class Solution {
2      int res=-0x3f3f3f3f;
3  public:
4      int maxgain(TreeNode* root){
5          if (root==NULL)
6              return 0;
7          int maxleft=max(maxgain(root->left),0);
8          int maxright=max(maxgain(root->right),0);
9          int curres=maxleft+maxright+root->val;
10         res=max(curres,res);
11         return root->val+max(maxleft,maxright);
12     }
13     int maxPathSum(TreeNode* root) {
14         maxgain(root);
15         return res;
16     }
17 };

```

AI写代码cpp运行

128. 最长连续序列

给定一个未排序的整数数组 *nums* ，找出数字连续的最长序列（不要求序列元素在原数组中连续）的长度。

请你设计并实现时间复杂度为 $O(n)$ 的算法解决此问题。

```

1  class Solution {
2  public:
3      int longestConsecutive(vector<int>& nums) {
4          set<int> st;
5          for(auto num:nums)
6              st.insert(num);
7          int res=0;
8          for(auto num:nums){
9              if (st.count(num)&&!st.count(num-1)){
10                 int x=num;
11                 int y=x;
12                 st.erase(x);
13                 while(st.count(y+1)){
14                     y++;
15                     st.erase(y);
16                 }
17                 res=max(res,y-x+1);
18             }
19         }
20         return res;
21     }
22 };

```

AI写代码cpp运行

136. 只出现一次的数字

给定一个非空整数数组，除了某个元素只出现一次以外，其余每个元素均出现两次。找出那个只出现了一次的元素。

```

1  class Solution {
2  public:
3      int singleNumber(vector<int>& nums) {
4

```

```

5         int res=0;
6         for(auto num:nums){
7             res^=num;
8         }
9         return res;
10    }
};

```

AI写代码cpp运行

139. 单词拆分

给你一个字符串 s 和一个字符串列表 `wordDict` 作为字典。请你判断是否可以利用字典中出现的单词拼接出 s 。

注意：不要求字典中出现的单词全部都使用，并且字典中的单词可以重复使用。

```

1  class Solution {
2  public:
3      bool wordBreak(string s, vector<string>& wordDict) {
4          int n=s.size();
5          vector<int> f(n+1);
6          f[0]=1;
7          for(int i=1;i<=n;i++){
8              for(string word:wordDict){
9                  int len=word.size();
10                 if (i>=len){
11                     if (s.substr(i-len,len)==word)
12                         f[i]=f[i-len];
13                 }
14             }
15         }
16         return f[n];
17     }
18 };

```

AI写代码cpp运行

141. 环形链表

给你一个链表的头节点 `head`，判断链表中是否有环。

```

1  class Solution {
2  public:
3      bool hasCycle(ListNode *head) {
4          if (!head || !head->next) return 0;
5          ListNode *first = head, *second = first->next;
6
7          while (first && second)
8          {
9              if (first == second) return true;
10             first = first->next;
11             second = second->next;
12             if (second) second = second->next;
13         }
14
15         return false;
16     }
17 };

```

AI写代码cpp运行

142. 环形链表 II

给定一个链表的头节点 `head`，返回链表开始入环的第一个节点。如果链表无环，则返回 `null`。

```

1  class Solution {
2  public:
3      ListNode *detectCycle(ListNode *head) {
4          set<ListNode*> st;
5          while(head){
6              if (st.count(head))
7                  return head;
8              st.insert(head);
9          }

```

```

-         head=head->next;
10     }
11     return NULL;
12 }
13 };
AI写代码cpp运行

```

146. LRU 缓存

请你设计并实现一个满足 LRU (最近最少使用) 缓存 约束的数据结构。

实现 LRUCache 类：

LRUCache(int capacity) 以 正整数 作为容量 *capacity* 初始化 LRU 缓存

int get(int key) 如果关键字 *key* 存在于缓存中，则返回关键字的值，否则返回 -1 。

void put(int key, int value) 如果关键字 *key* 已经存在，则变更其数据值 *value* ； 如果不存在，则向缓存中插入该组 key-value 。如果插入操作导致关键字 *capacity* ， 则应该 逐出 最久未使用的关键字。

函数 get 和 put 必须以 $O(1)$ 的平均时间复杂度运行。

```

1  class LRUCache {
2  public:
3      struct Node{
4          int key,val;
5          Node *left,*right;
6          Node():key(0),val(0),left(NULL),right(NULL){}
7      };
8      Node *hu,*hr,*tu,*tr;
9      int n;
10     unordered_map<int,Node *>hash;
11     void delete_node(Node *p){
12         p->left->right=p->right;
13         p->right->left=p->left;
14     }
15     void insert_node(Node *head,Node *p){
16         p->right=head->right;
17         head->right=p;
18         p->left=head;
19         p->right->left=p;
20     }
21     LRUCache(int capacity) {
22         n=capacity;
23         hu=new Node(),hr=new Node(),tu=new Node(),tr=new Node();
24         hu->right=tu,tu->left=hu;
25         hr->right=tr,tr->left=hr;
26         for(int i=0;i<n;i++){
27             Node *p=new Node();
28             insert_node(hr,p);
29         }
30     }
31
32     int get(int key) {
33         if (hash[key]){
34             Node *p=hash[key];
35             delete_node(p);
36             insert_node(hu,p);
37             return p->val;
38         }
39         return -1;
40     }
41
42     void put(int key, int value) {
43         if (hash[key]){
44             Node *p=hash[key];
45             delete_node(p);
46             insert_node(hu,p);
47             p->val=value;
48             return;
49         }
50         if (!n){
51             n++;
52             Node *p=tu->left;
53             hash[p->key]=0;

```

```

54         delete_node(p);
55         insert_node(hr,p);
56     }
57     n--;
58     Node *p=hr->right;
59     p->key=key,p->val=value,hash[key]=p;
60     delete_node(p);
61     insert_node(hu,p);
62 }
63 };
64
65 /**
66  * Your LRUCache object will be instantiated and called as such:
67  * LRUCache* obj = new LRUCache(capacity);
68  * int param_1 = obj->get(key);
69  * obj->put(key,value);
70  */

```

AI写代码cpp运行

148. 排序链表

给你链表的头结点 head，请将其按升序排列并返回排序后的链表。

```

1  class Solution {
2  public:
3      ListNode* sortList(ListNode* head) {
4          if(head==NULL)
5              return head;
6          int n=0;
7          for(auto p=head;p=p->next)
8              n++;
9          auto dummy=new ListNode (-1);
10         dummy->next=head;
11         for(int i=1;i<n;i*=2){
12             auto cur=dummy;
13             for(int j=1;j+i<=n;j+=i*2){
14                 auto p=cur->next,q=p;
15                 for(int k=0;k<i;k++){
16                     q=q->next;
17                     int x=0,y=0;
18                     while(x<i&&y<i&&p&&q){
19                         if (p->val<=q->val){
20                             cur=cur->next=p;
21                             p=p->next;
22                             x++;
23                         }
24                         else{
25                             cur=cur->next=q;
26                             q=q->next;
27                             y++;
28                         }
29                     }
30                     while(x<i&&p){
31                         cur=cur->next=p;
32                         p=p->next;
33                         x++;
34                     }
35                     while(y<i&&q){
36                         cur=cur->next=q;
37                         q=q->next;
38                         y++;
39                     }
40                     cur->next=q;
41                 }
42             }
43             return dummy->next;
44         }
45     };

```

AI写代码cpp运行

152. 乘积最大子数组

给你一个整数数组 *nums*，请你找出数组中乘积最大的非空连续子数组（该子数组中至少包含一个数字），并返回该子数组所对应的乘积。

```
1  class Solution {
2  public:
3      int maxProduct(vector<int>& nums) {
4          int res=nums[0],f=nums[0],g=nums[0];
5          for(int i=1;i<nums.size();i++){
6              int num=nums[i];
7              int fa=f*num;
8              int ga=g*num;
9              f=max(num,max(fa,ga));
10             g=min(num,min(fa,ga));
11             res=max(res,f);
12         }
13         return res;
14     }
15 };
```

AI写代码cpp运行

155. 最小栈

设计一个支持 push，pop，top 操作，并能在常数时间内检索到最小元素的栈。

实现 MinStack 类:

MinStack() 初始化堆栈对象。

void push(int val) 将元素val推入堆栈。

void pop() 删除堆栈顶部的元素。

int top() 获取堆栈顶部的元素。

int getMin() 获取堆栈中的最小元素。

```
1  class MinStack {
2      stack<int> stk;
3      stack<int> minstk;
4  public:
5      MinStack() {
6          minstk.push(INT_MAX);
7      }
8
9      void push(int val) {
10         stk.push(val);
11         if (val<=minstk.top())
12             minstk.push(val);
13     }
14
15     void pop() {
16         if (stk.top()==minstk.top())
17             minstk.pop();
18         stk.pop();
19     }
20
21     int top() {
22         return stk.top();
23     }
24
25     int getMin() {
26         if (minstk.size()==1)
27             return 0;
28         return minstk.top();
29     }
30 };
31
32 /**
33  * Your MinStack object will be instantiated and called as such:
34  * MinStack* obj = new MinStack();
35  * obj->push(val);
36  * obj->pop();
37  * int param_3 = obj->top();
38  * int param_4 = obj->getMin();
39  */
```

160. 相交链表

给你两个单链表的头节点 headA 和 headB，请你找出并返回两个单链表相交的起始节点。如果两个链表不存在相交节点，返回 null。

```

1  class Solution {
2  public:
3      ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
4          int len1=0,len2=0;
5          ListNode *p1=headA;
6          while(p1->next!=NULL){
7              len1++;
8              p1=p1->next;
9          }
10         ListNode *p2=headB;
11         while(p2->next!=NULL){
12             len2++;
13             p2=p2->next;
14         }
15         if (len1>len2)
16             return getIntersectionNode(headB,headA);
17         p2=headB;
18         p1=headA;
19         for(int i=0;i<len2-len1;i++){
20             p2=p2->next;
21         }
22         while(p1!=NULL&& p2!=NULL&&p1!=p2){
23             p1=p1->next;
24             p2=p2->next;
25         }
26         if (p1==p2)
27             return p1;
28         else return NULL;
29     };

```

AI写代码cpp运行

169. 多数元素

给定一个大小为 n 的数组 $nums$ ，返回其中的多数元素。多数元素是指在数组中出现次数 大于 $\lfloor n/2 \rfloor$ 的元素。你可以假设数组是非空的，并且给定的数组总是存在多数元素。

```

1  class Solution {
2  public:
3      int majorityElement(vector<int>& nums) {
4          int r=0,c=0;
5          for(auto num:nums){
6              if (c==0)
7                  r=num;
8              if (num!=r)
9                  c--;
10             else c++;
11         }
12         return r;
13     }
14 };

```

AI写代码cpp运行

198. 打家劫舍

你是一个专业的小偷，计划偷窃沿街的房屋。每间房内都藏有一定的现金，影响你偷窃的唯一制约因素就是相邻的房屋装有相互连通的防盗系统，如果两间相邻的房屋在同一晚上被小偷闯入，系统会自动报警。

给定一个代表每个房屋存放金额的非负整数数组，计算你不触动警报装置的情况下，一夜之内能够偷窃到的最高金额。

```

1  class Solution {
2  public:
3      int dp[105];
4      int rob(vector<int>& nums) {
5          int len=nums.size();

```

```

6         if (len==0)
7             return 0;
8         if (len == 1)
9             return nums[0];
10        dp[0]=nums[0];
11        dp[1]=max(nums[0],nums[1]);
12        for(int i=2;i<len;i++)
13            dp[i]=max(dp[i-1],dp[i-2]+nums[i]);
14        return dp[len-1];
15    }
16};

```

AI写代码cpp运行

200. 岛屿数量

给你一个由 '1'（陆地）和 '0'（水）组成的二维网格，请你计算网格中岛屿的数量。

岛屿总是被水包围，并且每座岛屿只能由水平方向和/或竖直方向上相邻的陆地连接形成。

此外，你可以假设该网格的四条边均被水包围。

```

1  class Solution {
2  public:
3      int dx[4]={-1,0,1,0};
4      int dy[4]={0,1,0,-1};
5      int numIslands(vector<vector<char>>& grid) {
6          int m=grid.size();
7          int n=grid[0].size();
8          vector<vector<bool>> st(m,vector<bool>(n,0));
9          int res=0;
10         for(int i=0;i<m;i++){
11             for(int j=0;j<n;j++){
12                 if (!st[i][j]&&grid[i][j]=='1'){
13                     st[i][j]=1;
14                     res++;
15                     queue<pair<int,int>> q;
16                     q.push({i,j});
17                     while(!q.empty()){
18                         auto [px,py]=q.front();
19                         q.pop();
20                         int tx,ty;
21                         for (int k=0;k<4;k++){
22                             tx=px+dx[k];
23                             ty=py+dy[k];
24                             if (tx<0||tx>=m||ty<0||ty>=n)
25                                 continue;
26                             if (st[tx][ty])
27                                 continue;
28                             if (grid[tx][ty]!='1')
29                                 continue;
30                             q.push({tx,ty});
31                             st[tx][ty]=1;
32                         }
33                     }
34                 }
35             }
36         }
37         return res;
38     }
39 };

```

AI写代码cpp运行

206. 反转链表

给你单链表的头节点 head ，请你反转链表，并返回反转后的链表。

```

1  class Solution {
2  public:
3      ListNode* reverseList(ListNode* head) {
4          if (head==NULL)
5              return head;
6          ListNode *p2=head->next;
7

```



```

1      ListNode *p1=head;
2      p1->next=NULL;
3      while(p2!=NULL){
4          ListNode *ptemp=p2->next;
5          p2->next=p1;
6          p1=p2;
7          p2=ptemp;
8      }
9      return p1;
10     }
11 }
12 };

```

AI写代码cpp运行

207. 课程表

你这个学期必须选修 numCourses 门课程，记为 0 到 numCourses - 1。

在选修某些课程之前需要一些先修课程。先修课程按数组 prerequisites 给出，其中 prerequisites[i] = [ai, bi]，表示如果要学习课程 ai 则必须先学习课程 bi。例如，先修课程对 [0, 1] 表示：想要学习课程 0，你需要先完成课程 1。

请你判断是否可能完成所有课程的学习？如果可以，返回 true；否则，返回 false。

```

1  class Solution {
2      vector<int> G[100005];
3      int indegree[100005];
4  public:
5      bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
6          int n=numCourses;
7          queue<int> q;
8          for (auto node:prerequisites){
9              G[node[0]].push_back(node[1]);
10             }
11             memset(indegree,0,sizeof indegree);
12             for(int i=0;i<n;i++){
13                 for(auto node:G[i])
14                     indegree[node]++;
15             int count=0;
16             for (int i=0;i<n;i++){
17                 if (indegree[i]==0)
18                     q.push(i);
19                 while(!q.empty()){
20                     int p=q.front();
21                     q.pop();
22                     count++;
23                     for (auto node:G[p]){
24                         indegree[node]--;
25                         if (indegree[node]==0){
26                             q.push(node);
27                         }
28                     }
29                 }
30                 return count==n;
31             }
32     };

```

AI写代码cpp运行

208. 实现 Trie (前缀树)

```

1  class Trie {
2  public:
3      bool isword[300005];
4      int ne[300005][26];
5      int idx=0;
6      Trie() {
7          memset(isword,0,sizeof isword);
8          idx=0;
9          memset(ne,0,sizeof ne);
10     }
11     void insert(string word) {
12         int n=word.size();
13         int p=0;
14         for(int i=0;i<n;i++){

```

```

15         int u=word[i]-'a';
16         if (!ne[p][u]){
17             ne[p][u]=++idx;
18         }
19         p=ne[p][u];
20     }
21     isword[p]=1;
22 }
23 bool search(string word) {
24     int n=word.size();
25     int p=0;
26     for(int i=0;i<n;i++){
27         int u=word[i]-'a';
28         if (!ne[p][u])
29             return false;
30         p=ne[p][u];
31     }
32     return isword[p];
33 }
34 bool startsWith(string prefix) {
35     int n=prefix.size();
36     int p=0;
37     for(int i=0;i<n;i++){
38         int u=prefix[i]-'a';
39         if (!ne[p][u])
40             return false;
41         p=ne[p][u];
42     }
43     return true;
44 }
45 };
46 /**
47  * Your Trie object will be instantiated and called as such:
48  * Trie* obj = new Trie();
49  * obj->insert(word);
50  * bool param_2 = obj->search(word);
51  * bool param_3 = obj->startsWith(prefix);
52  */

```

AI写代码cpp运行

215. 数组中的第K个最大元素

给定整数数组 *nums* 和整数 *k*，请返回数组中第 *k* 个最大的元素。

请注意，你需要找的是数组排序后的第 *k* 个最大的元素，而不是第 *k* 个不同的元素。

你必须设计并实现时间复杂度为 $O(n)$ 的算法解决此问题。

```

1  class Solution {
2  public:
3      int quick_selection(vector<int>& nums,int l,int r,int k){
4          if (l==r)
5              return nums[k];
6          int x=nums[l];
7          int i=l-1,j=r+1;
8          while(i<j){
9              do i++;while(nums[i]<x);
10             do j--;while(nums[j]>x);
11             if (i<j)
12                 swap(nums[i],nums[j]);
13         }
14         if (k<=j)
15             return quick_selection(nums,l,j,k);
16         else return quick_selection(nums,j+1,r,k);
17     }
18     int findKthLargest(vector<int>& nums, int k) {
19         return quick_selection(nums,0,nums.size()-1,nums.size()-k);
20     }
21 };

```

AI写代码cpp运行

221. 最大正方形

在一个由 '0' 和 '1' 组成的二维矩阵内，找到只包含 '1' 的最大正方形，并返回其面积。

```
1 class Solution {
2 public:
3     int maximalSquare(vector<vector<char>>& matrix) {
4         int n=matrix.size();
5         int m=matrix[0].size();
6         int res=0;
7         vector<vector<int>> f(n,vector<int>(m,0));
8         for(int i=0;i<n;i++){
9             for(int j=0;j<m;j++){
10                 if (i==j&& i==0)
11                     f[i][j]=matrix[i][j]-'0';
12                 else if (i==0||j==0)
13                     f[i][j]=matrix[i][j]-'0';
14                 else if (matrix[i][j]=='1')
15                     f[i][j]=min(f[i-1][j],min(f[i][j-1],f[i-1][j-1]))+1;
16                 res=max(res,f[i][j]);
17             }
18         }
19         return res*res;
20     }
21 };
AI写代码cpp运行
```

226. 翻转二叉树

```
1 class Solution {
2 public:
3     TreeNode* invertTree(TreeNode* root) {
4         if (root==NULL)
5             return root;
6         if (root->left!=NULL)
7             invertTree(root->left);
8         if (root->right!=NULL)
9             invertTree(root->right);
10        swap(root->left,root->right);
11        return root;
12    }
13 };
AI写代码cpp运行
```

234. 回文链表

```
1 class Solution {
2 public:
3     bool isPalindrome(ListNode* head) {
4         stack<int> stk;
5         ListNode *p=head;
6         while(p!=NULL){
7             stk.push(p->val);
8             p=p->next;
9         }
10        p=head;
11        while(!stk.empty()){
12            int t=stk.top();
13            stk.pop();
14            if (t!=p->val){
15                return false;
16            }
17            p=p->next;
18        }
19        return true;
20    }
21 };
AI写代码cpp运行
```

236. 二叉树的最近公共祖先

给定一个二叉树, 找到该树中两个指定节点的最近公共祖先。

百度百科中最近公共祖先的定义为：“对于有根树 T 的两个节点 p 、 q ，最近公共祖先表示为一个节点 x ，满足 x 是 p 、 q 的祖先且 x 的深度尽可能大（一个节点的祖先）。”

```
1 class Solution {
2 public:
3     unordered_map<TreeNode *,int> l;
4     unordered_map<TreeNode *,TreeNode *>f;
5     void dfs(TreeNode *root){
6         if (root->left){
7             f[root->left]=root;
8             l[root->left]=l[root]+1;
9             dfs(root->left);
10        }
11        if (root->right){
12            f[root->right]=root;
13            l[root->right]=l[root]+1;
14            dfs(root->right);
15        }
16        return;
17    }
18    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
19        f[root]=NULL;
20        l[root]=0;
21        dfs(root);
22        int lp=l[p];
23        int lq=l[q];
24        while(lp>lq){
25            p=f[p];
26            lp=l[p];
27        }
28        while(lq>lp){
29            q=f[q];
30            lq=l[q];
31        }
32        while(p!=q){
33            p=f[p];
34            q=f[q];
35        }
36        return p;
37    }
38};
```

AI写代码cpp运行

238. 除自身以外数组的乘积

```
1 class Solution {
2 public:
3     vector<int> productExceptSelf(vector<int>& nums) {
4         int n=nums.size();
5         vector<int> l(n,1);//左缀
6         vector<int> r(n,1);//右缀
7         for(int i=1;i<n;i++)
8             l[i]=l[i-1]*nums[i-1];
9         for(int i=n-2;i>=0;i--)
10            r[i]=r[i+1]*nums[i+1];
11        vector<int> res(n,1);
12        for(int i=0;i<n;i++)
13            res[i]=l[i]*r[i];
14        return res;
15    }
16};
```

AI写代码cpp运行

239. 滑动窗口最大值

给你一个整数数组 $nums$ ，有一个大小为 k 的滑动窗口从数组的最左侧移动到数组的最右侧。你只可以看到在滑动窗口内的 k 个数字。滑动窗口每次只向右返回 滑动窗口中的最大值。

```

1 class Solution {
2 public:
3     vector<int> maxSlidingWindow(vector<int>& nums, int k) {
4         deque<int> dq;
5         int n=nums.size();
6         vector<int> ans;
7         for (int i = 0; i < n; i++) {
8             if (!dq.empty() && i - k + 1 > dq.front())
9                 dq.pop_front();
10            while (!dq.empty() && nums[dq.back()] <= nums[i])
11                dq.pop_back();
12            dq.push_back(i);
13            if (i >= k - 1)
14                ans.push_back(nums[dq.front()]);
15        }
16        return ans;
17    }
18 };

```

AI写代码cpp运行

240. 搜索二维矩阵 II

编写一个高效的算法来搜索 $m \times n$ 矩阵 $matrix$ 中的一个目标值 $target$ 。该矩阵具有以下特性：

每行的元素从左到右升序排列。

每列的元素从上到下升序排列。

```

1 class Solution {
2 public:
3     bool searchMatrix(vector<vector<int>>& matrix, int target) {
4         if (matrix.empty() || matrix[0].empty())
5             return false;
6         int n=matrix.size();
7         int m=matrix[0].size();
8         int i=0,j=m-1;
9         while(i<n&&j>=0){
10            int t=matrix[i][j];
11            if (t==target)
12                return true;
13            else if (t>target)
14                j--;
15            else i++;
16        }
17        return false;
18    }
19 };

```

AI写代码cpp运行

279. 完全平方数

给你一个整数 n ，返回和为 n 的完全平方数的最少数量。

完全平方数 是一个整数，其值等于另一个整数的平方；换句话说，其值等于一个整数自乘的积。例如，1、4、9 和 16 都是完全平方数，而 3 和 11 不是。

```

1 class Solution {
2 public:
3     int numSquares(int n) {
4         vector<int> f(n+1,0x3f3f3f3f);
5         f[0]=0;
6         f[1]=1;
7         for(int i=2;i<=n;i++){
8             for(int j=1;j*j<=i;j++){
9                 f[i]=min(f[i],f[i-j*j]+1);
10            }
11            return f[n];
12        }
13    };

```

AI写代码cpp运行

283. 移动零

给定一个数组 `nums`，编写一个函数将所有 0 移动到数组的末尾，同时保持非零元素的相对顺序。
请注意，必须在复制数组的情况下原地对数组进行操作。

```
1 class Solution {
2 public:
3     void moveZeroes(vector<int>& nums) {
4         int n=nums.size();
5         int i=0,j=0;
6         for(;i<n;i++){
7             if (nums[i]!=0){
8                 nums[j]=nums[i];
9                 j++;
10            }
11        }
12        for(;j<n;j++)
13            nums[j]=0;
14    }
15};
```

AI写代码cpp运行

287. 寻找重复数

给定一个包含 $n + 1$ 个整数的数组 `nums`，其数字都在 $[1, n]$ 范围内（包括 1 和 n ），可知至少存在一个重复的整数。
假设 `nums` 只有一个重复的整数，返回这个重复的数。
你设计的解决方案必须 不修改 数组 `nums` 且只用常量级 $O(1)$ 的额外空间。

```
1 class Solution {
2 public:
3     int findDuplicate(vector<int>& nums) {
4         int slow=0,fast=0;
5         do{
6             slow=nums[slow];
7             fast=nums[nums[fast]];
8         }while(slow!=fast);
9         slow=0;
10        while(slow!=fast){
11            slow=nums[slow];
12            fast=nums[fast];
13        }
14        return slow;
15    }
16};
```

AI写代码cpp运行

297. 二叉树的序列化与反序列化

```
1 class Codec {
2 public:
3     string path;
4     // Encodes a tree to a single string.
5     string serialize(TreeNode* root) {
6         dfs_s(root);
7         return path;
8     }
9     void dfs_s(TreeNode* root) {
10        if (!root) path += "#,";
11        else {
12            path += to_string(root->val) + ',';
13            dfs_s(root->left);
14            dfs_s(root->right);
15        }
16    }
17    // Decodes your encoded data to tree.
18    TreeNode* deserialize(string data) {
19        int u = 0;
20        return dfs_d(data, u);
21    }
22    TreeNode* dfs_d(string& data, int& u) {
23        if (data[u] == '#') {
24
```

```

25         u += 2;
26         return NULL;
27     } else {
28         int k = u;
29         while (data[u] != ',') u ++ ;
30         auto root = new TreeNode(stoi(data.substr(k, u - k)));
31         u ++ ;
32         root->left = dfs_d(data, u);
33         root->right = dfs_d(data, u);
34         return root;
35     }
36 }
};
AI写代码cpp运行

```

301. 删除无效的括号

给你一个由若干括号和字母组成的字符串 *s*，删除最小数量的无效括号，使得输入的字符串有效。返回所有可能的结果。答案可以按 任意顺序 返回。

```

1  class Solution {
2  public:
3      vector<string> ans;
4      vector<string> removeInvalidParentheses(string s) {
5          int l = 0, r = 0;
6          for (auto x: s)
7              if (x == '(') l ++ ;
8              else if (x == ')') {
9                  if (l == 0) r ++ ;
10                 else l -- ;
11             }
12         dfs(s, 0, "", 0, l, r);
13         return ans;
14     }
15     void dfs(string& s, int u, string path, int cnt, int l, int r) {
16         if (u == s.size()) {
17             if (!cnt) ans.push_back(path);
18             return;
19         }
20         if (s[u] != '(' && s[u] != ')') dfs(s, u + 1, path + s[u], cnt, l, r);
21         else if (s[u] == '(') {
22             int k = u;
23             while (k < s.size() && s[k] == '(') k ++ ;
24             l -= k - u;
25             for (int i = k - u; i >= 0; i -- ) {
26                 if (l >= 0) dfs(s, k, path, cnt, l, r);
27                 path += '(';
28                 cnt ++, l ++ ;
29             }
30         } else if (s[u] == ')') {
31             int k = u;
32             while (k < s.size() && s[k] == ')') k ++ ;
33             r -= k - u;
34             for (int i = k - u; i >= 0; i -- ) {
35                 if (cnt >= 0 && r >= 0) dfs(s, k, path, cnt, l, r);
36                 path += ')';
37                 cnt --, r ++ ;
38             }
39         }
40     }
41 };
AI写代码cpp运行

```

309. 最佳买卖股票时机含冷冻期

给定一个整数数组 *prices*，其中第 *prices[i]* 表示第 *i* 天的股票价格。

设计一个算法计算出最大利润。在满足以下约束条件下，你可以尽可能地完成更多的交易（多次买卖一支股票）：

卖出股票后，你无法在第二天买入股票（即冷冻期为 1 天）。

注意：你不能同时参与多笔交易（你必须在再次购买前出售掉之前的股票）。

```

1 class Solution {
2 public:
3     int maxProfit(vector<int>& prices) {
4         int n=prices.size();
5         vector<vector<int>> f(n,vector<int>(3,0));
6         f[0][1]=-prices[0];
7         int res=0;
8         for(int i=1;i<n;i++){
9             f[i][0]=max(f[i-1][0],f[i-1][2]);
10            f[i][1]=max(f[i-1][0]-prices[i],f[i-1][1]);
11            f[i][2]=f[i-1][1]+prices[i];
12        }
13        return max(f[n-1][0],f[n-1][2]);
14    }
15 };

```

AI写代码cpp运行

312. 戳气球

有 n 个气球，编号为 0 到 $n-1$ ，每个气球上都标有一个数字，这些数字存在数组 $nums$ 中。

现在要求你戳破所有的气球。戳破第 i 个气球，你可以获得 $nums[i-1] * nums[i] * nums[i+1]$ 枚硬币。这里的 $i-1$ 和 $i+1$ 代表和 i 相邻的两个气球。如果 $i-1$ 或 $i+1$ 超出了数组的边界，那么就当它是一个数字为 1 的气球。

求所能获得硬币的最大数量。

```

1 class Solution {
2 public:
3     int maxCoins(vector<int>& nums) {
4         int n=nums.size();
5         vector<int> a(n+2,1);
6         for(int i=0;i<n;i++)
7             a[i+1]=nums[i];
8         vector<vector<int>> f(n+2,vector<int>(n+2,0));
9         for(int len=3;len<=n+2;len++){
10            for(int i=0;i+len-1<=n+1;i++){
11                int j=i+len-1;
12                for(int k=i+1;k<j;k++){
13                    f[i][j]=max(f[i][j],f[i][k]+f[k][j]+a[i]*a[k]*a[j]);
14                }
15            }
16        }
17        return f[0][n+1];
18    }
19 };

```

AI写代码cpp运行

322. 零钱兑换

给你一个整数数组 $coins$ ，表示不同面额的硬币；以及一个整数 $amount$ ，表示总金额。

计算并返回可以凑成总金额所需的最少的硬币个数。如果没有任何一种硬币组合能组成总金额，返回 -1 。

你可以认为每种硬币的数量是无限的。

```

1 class Solution {
2 public:
3     int coinChange(vector<int>& coins, int amount) {
4         sort(coins.begin(),coins.end());
5         vector<int> f(amount+1,0x3f3f3f3f);
6         f[0]=0;
7         for(int i=1;i<=amount;i++){
8             for(int coin:coins){
9                 if (i>=coin)
10                    f[i]=min(f[i],f[i-coin]+1);
11            }
12        }
13        if (f[amount]==0x3f3f3f3f)
14            return -1;
15        else return f[amount];
16    }
17 };

```

AI写代码cpp运行

337. 打家劫舍 III

小偷又发现了一个新的可行窃的地区。这个地区只有一个入口，我们称之为 root。

除了 root 之外，每栋房子有且只有一个“父”房子与之相连。一番侦察之后，聪明的小偷意识到“这个地方的所有房屋的排列类似于一棵二叉树”。如果两个在同一晚上被打劫，房屋将自动报警。

给定二叉树的 root。返回 在不触动警报的情况下，小偷能够盗取的最高金额。

```
1 class Solution {
2 public:
3     vector<int> dfs(TreeNode *root){
4         if (root==NULL)
5             return {0,0};
6         vector<int> f(2);
7         auto l=dfs(root->left);
8         auto r=dfs(root->right);
9         f[0]=max(l[0],l[1])+max(r[0],r[1]);
10        f[1]=l[0]+r[0]+root->val;
11        return f;
12    }
13    int rob(TreeNode* root) {
14        auto f=dfs(root);
15        return max(f[0],f[1]);
16    }
17 };
AI写代码cpp运行
```

338. 比特位计数

给你一个整数 n ，对于 $0 \leq i \leq n$ 中的每个 i ，计算其二进制表示中 1 的个数，返回一个长度为 $n + 1$ 的数组 *ans* 作为答案。

```
1 class Solution {
2 public:
3     vector<int> countBits(int n) {
4         vector<int> f(n+1);
5         for(int i=1;i<=n;i++){
6             f[i]=f[i>>1]+(i&1);
7         }
8         return f;
9     }
10 };
AI写代码cpp运行
```

347. 前 K 个高频元素

给你一个整数数组 *nums* 和一个整数 *k*，请你返回其中出现频率前 *k* 高的元素。你可以按 任意顺序 返回答案。

```
1 class Solution {
2 public:
3     vector<int> topKFrequent(vector<int>& nums, int k) {
4         unordered_map<int, int> cnt;
5         for (auto x: nums) cnt[x] ++ ;
6         int n = nums.size();
7         vector<int> s(n + 1);
8         for (auto [x, c]: cnt) s[c] ++ ;
9         int i = n, t = 0;
10        while (t < k) t += s[i -- ];
11        vector<int> res;
12        for (auto [x, c]: cnt)
13            if (c > i)
14                res.push_back(x);
15        return res;
16    }
17 };
AI写代码cpp运行
```

394. 字符串解码

```
1 class Solution {
2 public:
3     string decodeString(string s) {
```

```

4         int u = 0;
5         return dfs(s, u);
6     }
7
8     string dfs(string& s, int& u) {
9         string res;
10        while (u < s.size() && s[u] != ']') {
11            if (s[u] >= 'a' && s[u] <= 'z' || s[u] >= 'A' && s[u] <= 'Z') res += s[u++];
12            else if (s[u] >= '0' && s[u] <= '9') {
13                int k = u;
14                while (s[k] >= '0' && s[k] <= '9') k++;
15                int x = stoi(s.substr(u, k - u));
16                u = k + 1;
17                string y = dfs(s, u);
18                u++; // 过滤掉右括号
19                while (x-- > 0) res += y;
20            }
21        }
22        return res;
23    }
24 };

```

AI写代码cpp运行

399. 除法求值

给你一个变量对数组 *equations* 和一个实数值数组 *values* 作为已知条件，其中 $equations_i = A_i, B_i$ 和 $values_i$ 共同表示等式 $A_i / B_i = values_i$ 。每个表示单个变量的字符串。

另有一些以数组 *queries* 表示的问题，其中 $queries_j = C_j, D_j$ 表示第 *j* 个问题，请你根据已知条件找出 $C_j / D_j = ?$ 的结果作为答案。

返回 所有问题的答案。如果存在某个无法确定的答案，则用 -1.0 替代这个答案。如果问题中出现了给定的已知条件中没有出现的字符串，也需要用 -1.0 替代。注意：输入总是有效的。你可以假设除法运算中不会出现除数为 0 的情况，且不存在任何矛盾的结果。

```

1 class Solution {
2 public:
3     vector<double> calcEquation(vector<vector<string>>& equations, vector<double>& values, vector<vector<string>>& queries) {
4         unordered_set<string> vers;
5         unordered_map<string, unordered_map<string, double>> d;
6         for (int i = 0; i < equations.size(); i++) {
7             auto a = equations[i][0], b = equations[i][1];
8             auto c = values[i];
9             d[a][b] = c, d[b][a] = 1 / c;
10            vers.insert(a), vers.insert(b);
11        }
12        for (auto k: vers)
13            for (auto i: vers)
14                for (auto j: vers)
15                    if (d[i][k] && d[j][k])
16                        d[i][j] = d[i][k] * d[k][j];
17        vector<double> res;
18        for (auto q: queries) {
19            auto a = q[0], b = q[1];
20            if (d[a][b]) res.push_back(d[a][b]);
21            else res.push_back(-1);
22        }
23        return res;
24    }
25 };

```

AI写代码cpp运行

402. 移掉K位数字

```

1 class Solution {
2 public:
3     string removeKdigits(string num, int k) {
4         k = min(k, (int)num.size());
5         string res;
6         for (auto c: num) {
7             while (k && res.size() && res.back() > c) {
8                 k--;
9                 res.pop_back();
10            }
11        }
12    }
13 };

```

```

11         res += c;
12     }
13     while (k -- ) res.pop_back();
14     k = 0;
15     while (k < res.size() && res[k] == '0') k ++ ;
16     if (k == res.size()) res += '0';
17     return res.substr(k);
18 }
19 };

```

AI写代码cpp运行

406. 根据身高重建队列

假设有打乱顺序的一群人站成一个队列，数组 $people$ 表示队列中一些人的属性（不一定按顺序）。每个 $people_i = h_i, k_i$ 表示第 i 个人的身高为 h_i ，前面大于或等于 h_i 的人。

请你重新构造并返回输入数组 $people$ 所表示的队列。返回的队列应该格式化为数组 $queue$ ，其中 $queue_j = h_j, k_j$ 是队列中第 j 个人的属性（ $queue_0$ 是人）。

```

1  class Solution {
2  public:
3      int n;
4      vector<int> tr;
5      int lowbit(int x) {
6          return x & -x;
7      }
8      void add(int x, int v) {
9          for (int i = x; i <= n; i += lowbit(i)) tr[i] += v;
10     }
11     int query(int x) {
12         int res = 0;
13         for (int i = x; i; i -= lowbit(i)) res += tr[i];
14         return res;
15     }
16     vector<vector<int>> reconstructQueue(vector<vector<int>>& people) {
17         n = people.size();
18         tr.resize(n + 1);
19         sort(people.begin(), people.end(), [](vector<int>a, vector<int>b) {
20             if (a[0] != b[0]) return a[0] < b[0];
21             return a[1] > b[1];
22         });
23         vector<vector<int>> res(n);
24         for (auto p: people) {
25             int h = p[0], k = p[1];
26             int l = 1, r = n;
27             while (l < r) {
28                 int mid = l + r >> 1;
29                 if (mid - query(mid) >= k + 1) r = mid;
30                 else l = mid + 1;
31             }
32             res[r - 1] = p;
33             add(r, 1);
34         }
35         return res;
36     }
37 };
38 /*
39 class Solution {
40 public:
41     vector<vector<int>> reconstructQueue(vector<vector<int>>& people) {
42         auto cmp = [](const vector<int>& a, const vector<int>& b) {
43             if(a[0] != b[0]) return a[0] < b[0];
44             return a[1] > b[1];
45         };
46         sort(people.begin(), people.end(), cmp);
47         int n = people.size();
48         vector<vector<int>> res(n);
49         for(auto &p: people) {
50             int k = p[1] + 1;
51             for(auto &seg: res)
52                 if(seg.empty()) {
53

```

```

54         k--;
55         if(!k) {
56             seg = p;
57             break;
58         }
59     }
60 }
61 return res;
62 }
63 };
64 */

```

AI写代码cpp运行

416. 分割等和子集

给你一个 只包含正整数的非空数组 *nums* 请你判断是否可以将这个数组分割成两个子集，使得两个子集的元素和相等。

```

1  class Solution {
2  public:
3      bool canPartition(vector<int>& nums) {
4          bitset<10001> f;
5          f[0] = 1;
6          int sum = 0;
7          for (auto x: nums) {
8              f |= f << x;
9              sum += x;
10         }
11         if (sum % 2) return false;
12         return f[sum / 2];
13     }
14 };

```

AI写代码cpp运行

437. 路径总和 III

给定一个二叉树的根节点 *root*， 和一个整数 *targetSum*， 求该二叉树里节点值之和等于 *targetSum* 的路径的数目。路径不需要从根节点开始，也不需要叶子节点结束，但是路径方向必须是向下的（只能从父节点到子节点）。

```

1  class Solution {
2  public:
3      unordered_map<int, int> cnt;
4      int res = 0;
5      int pathSum(TreeNode* root, int sum) {
6          cnt[0] ++ ;
7          dfs(root, sum, 0);
8          return res;
9      }
10     void dfs(TreeNode* root, int sum, int cur) {
11         if (!root) return;
12         cur += root->val;
13         res += cnt[cur - sum];
14         cnt[cur] ++ ;
15         dfs(root->left, sum, cur), dfs(root->right, sum, cur);
16         cnt[cur] -- ;
17     }
18 };

```

AI写代码cpp运行

438. 找到字符串中所有字母异位词

给定两个字符串 *s* 和 *p*， 找到 *s* 中所有 *p* 的异位词的子串， 返回这些子串的起始索引。不考虑答案输出的顺序。异位词 指由相同字母重排列形成的字符串（包括相同的字符串）。

```

1  class Solution {
2  public:
3      vector<int> findAnagrams(string s, string p) {
4          unordered_map<char, int> cnt;
5          for (auto c: p) cnt[c] ++ ;
6          vector<int> res;
7          int tot = cnt.size();

```

```

8         for (int i = 0, j = 0, satisfy = 0; i < s.size(); i ++ ) {
9             if ( -- cnt[s[i]] == 0) satisfy ++ ;
10            while (i - j + 1 > p.size()) {
11                if (cnt[s[j]] == 0) satisfy -- ;
12                cnt[s[j ++ ]] ++ ;
13            }
14            if (satisfy == tot) res.push_back(j);
15        }
16        return res;
17    }
18 };

```

AI写代码cpp运行

448. 找到所有数组中消失的数字

给你一个含 n 个整数的数组 $nums$ ，其中 $nums_i$ 在区间 $[1, n]$ 内。请你找出所有在 $[1, n]$ 范围内但没有出现在 $nums$ 中的数字，并以数组的形式返回结果。

```

1 class Solution {
2 public:
3     vector<int> findDisappearedNumbers(vector<int>& nums) {
4         for (auto x: nums) {
5             x = abs(x);
6             if (nums[x - 1] > 0) nums[x - 1] *= -1;
7         }
8         vector<int> res;
9         for (int i = 0; i < nums.size(); i ++ )
10            if (nums[i] > 0)
11                res.push_back(i + 1);
12        return res;
13    }
14 };

```

AI写代码cpp运行

461. 汉明距离

```

1 class Solution {
2 public:
3     int hammingDistance(int x, int y) {
4         int res = 0;
5         while (x || y) {
6             res += (x & 1) ^ (y & 1);
7             x >>= 1, y >>= 1;
8         }
9         return res;
10    }
11 };

```

AI写代码cpp运行

494. 目标和

给你一个整数数组 $nums$ 和一个整数 $target$ 。

向数组中的每个整数前添加 '+' 或 '-'，然后串联起所有整数，可以构造一个表达式：

例如， $nums = [2, 1]$ ，可以在 2 之前添加 '+'，在 1 之前添加 '-'，然后串联起来得到表达式 "+2-1"。

返回可以通过上述方法构造的、运算结果等于 $target$ 的不同表达式的数目。

```

1 class Solution {
2 public:
3     int findTargetSumWays(vector<int>& a, int S) {
4         if (S < -1000 || S > 1000) return 0;
5         int n = a.size(), Offset = 1000;
6         vector<vector<int>> f(n + 1, vector<int>(2001));
7         f[0][Offset] = 1;
8         for (int i = 1; i <= n; i ++ )
9             for (int j = -1000; j <= 1000; j ++ ) {
10                if (j - a[i - 1] >= -1000)
11                    f[i][j + Offset] += f[i - 1][j - a[i - 1] + Offset];
12                if (j + a[i - 1] <= 1000)
13                    f[i][j + Offset] += f[i - 1][j + a[i - 1] + Offset];
14            }
15    }

```

```

16         return f[n][S + Offset];
17     }
};
AI写代码cpp运行

```

538. 把二叉搜索树转换为累加树

```

1  class Solution {
2  public:
3      int sum = 0;
4
5      TreeNode* convertBST(TreeNode* root) {
6          dfs(root);
7          return root;
8      }
9      void dfs(TreeNode* root) {
10         if (!root) return;
11         dfs(root->right);
12         int x = root->val;
13         root->val += sum;
14         sum += x;
15         dfs(root->left);
16     }
17 };
AI写代码cpp运行

```

543. 二叉树的直径

```

1  class Solution {
2  public:
3      int ans = 0;
4      int diameterOfBinaryTree(TreeNode* root) {
5          dfs(root);
6          return ans;
7      }
8      int dfs(TreeNode* root) {
9          if (!root) return 0;
10         int left = dfs(root->left), right = dfs(root->right);
11         ans = max(ans, left + right);
12         return max(left, right) + 1;
13     }
14 };
AI写代码cpp运行

```

560. 和为K的子数组

给你一个整数数组 $nums$ 和一个整数 k ，请你统计并返回该数组中和为 k 的连续子数组的个数。

```

1  class Solution {
2  public:
3      int subarraySum(vector<int>& nums, int k) {
4          int n = nums.size();
5          vector<int> s(n + 1);
6          for (int i = 1; i <= n; i++) s[i] = s[i - 1] + nums[i - 1];
7          unordered_map<int, int> hash;
8          hash[0] = 1;
9          int res = 0;
10         for (int i = 1; i <= n; i++) {
11             res += hash[s[i] - k];
12             hash[s[i]]++;
13         }
14         return res;
15     }
16 };
AI写代码cpp运行

```

581. 最短无序连续子数组

给你一个整数数组 $nums$ ，你需要找出一个连续子数组，如果对这个子数组进行升序排序，那么整个数组都会变为升序排序。请你找出符合题意的最短子数组，并输出它的长度。

```
1 class Solution {
2 public:
3     int findUnsortedSubarray(vector<int>& nums) {
4         int l = 0, r = nums.size() - 1;
5         while (l + 1 < nums.size() && nums[l + 1] >= nums[l]) l ++ ;
6         if (l == r) return 0;
7         while (r - 1 >= 0 && nums[r - 1] <= nums[r]) r -- ;
8         for (int i = l + 1; i < nums.size(); i ++ )
9             while (l >= 0 && nums[l] > nums[i])
10                 l -- ;
11         for (int i = r - 1; i >= 0; i -- )
12             while (r < nums.size() && nums[r] < nums[i])
13                 r ++ ;
14         return r - l - 1;
15     }
16 };
```

AI写代码cpp运行

617. 合并二叉树

```
1 class Solution {
2 public:
3     TreeNode* mergeTrees(TreeNode* t1, TreeNode* t2) {
4         if (t2) swap(t1, t2); // 可以保证t1一定不为空
5         if (!t1) return NULL;
6         if (t2) t1->val += t2->val;
7         t1->left = mergeTrees(t1->left, t2 ? t2->left : NULL);
8         t1->right = mergeTrees(t1->right, t2 ? t2->right : NULL);
9         return t1;
10    }
11 };
```

AI写代码cpp运行

621. 任务调度器

给你一个用字符串数组 $tasks$ 表示的CPU需要执行的任务列表。其中每个字母表示一种不同种类的任务。任务可以以任意顺序执行，并且每个任务都可以在1行完。在任何单位时间，CPU 可以完成一个任务，或者处于待命状态。

然而，两个相同种类的任务之间必须有长度为整数 n 的冷却时间，因此至少有连续 n 个单位时间内 CPU 在执行不同的任务，或者在待命状态。

你需要计算完成所有任务所需要的 最短时间。

```
1 class Solution {
2 public:
3     int leastInterval(vector<char>& tasks, int n) {
4         unordered_map<char, int> hash;
5         for (auto c: tasks) hash[c] ++ ;
6         int maxc = 0, cnt = 0;
7         for (auto [k, v]: hash) maxc = max(maxc, v);
8         for (auto [k, v]: hash)
9             if (maxc == v)
10                 cnt ++ ;
11         return max((int)tasks.size(), (maxc - 1) * (n + 1) + cnt);
12    }
13 };
```

AI写代码cpp运行

647. 回文子串

给你一个字符串 s ，请你统计并返回这个字符串中 回文子串 的数目。

回文字符串 是正着读和倒过来读一样的字符串。

子字符串 是字符串中的由连续字符组成的一个序列。

具有不同开始位置或结束位置的子串，即使是由相同的字符组成，也会被视作不同的子串。

```
1 class Solution {
2 public:
3     int countSubstrings(string s) {
4
```

```

5     int res = 0;
6     for (int i = 0; i < s.size(); i ++ ) {
7         // 枚举长度为奇数的情况
8         for (int j = i, k = i; j >= 0 && k < s.size(); j --, k ++ ) {
9             if (s[j] != s[k]) break;
10            res ++ ;
11        }
12        // 偶数情况
13        for (int j = i, k = i + 1; j >= 0 && k < s.size(); j --, k ++ ) {
14            if (s[j] != s[k]) break;
15            res ++ ;
16        }
17    }
18    return res;
19 }
};
AI写代码cpp运行

```

739. 每日温度

给定一个整数数组 *temperatures*，表示每天的温度，返回一个数组 *answer*，其中 *answer_i* 是指对于第 *i* 天，下一个更高温度出现在几天后。如果气温不再升高，请在该位置用 0 来代替。

```

1  class Solution {
2  public:
3      vector<int> dailyTemperatures(vector<int>& T) {
4          stack<int> stk;
5          vector<int> res(T.size());
6          for (int i = T.size() - 1; i >= 0; i -- ) {
7              while (stk.size() && T[i] >= T[stk.top()]) stk.pop();
8              if (stk.size()) res[i] = stk.top() - i;
9              stk.push(i);
10         }
11         return res;
12     }
13 };
AI写代码cpp运行

```